



**Volume Number:** 9  
**Issue Number:** 4  
**Column Tag:** Programmers' Challenge

## Programmers' Challenge

By Mike Scanlin, MacTech Magazine Regular Contributing Author

 **Note:** [Source code files](#) accompanying article are located on MacTech CD-ROM or source code disks.

### Rotated Bits

This month's challenge is to write a routine that quickly rotates a 1-bit deep bitMap 90 degrees clockwise. You are given pointers to a source bitMap and destination bitMap. Space for the destination pixels has already been allocated (the baseAddr field is valid) and the rowBytes and bounds fields have been set up for you. All you have to do is rotate the bits.

The function prototype you write is:

```
void RotateBitMapClockwise(srcBitMapPtr, dstBitMapPtr)
BitMap *srcBitMapPtr;
BitMap *dstBitMapPtr;
```

To check for correctness, you might want to write a test program that rotates the same bitMap four times and then diffs the result with the original (they should be identical). Have fun.

### February Winner

The winner of the "Insane Anglo Warlord" challenge is Jeremy Vineyard (Lawrence, KS) whose solution was the only one of the two I received which gave valid results (the other one gave intermediate strings that had a character not in the original strings). Remember kids: you have to play the game to have a chance at winning.

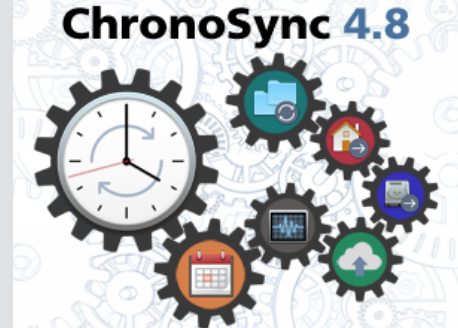
Here's the output of Jeremy's Descramble routine when the input strings are "INSANE ANGLO WARLORD" and "RONALD WILSON REAGAN" when the number of intermediate steps is set to 10:

```
INSANE ANGLO WARLORD
OIDSLE N NGWRALORAA
ORDLINE AWNSNLORAAG
R AODSILEWANN ORALGN
R WADOSGLNIAA RLOENN
R IWODNSALRAAL OEGNN
ROD INLASL RAWOEAGNN
RONDIL A SLORWAEAGNN
RONLDLI S AORWAEAGNN
ROND L WASIOR AEAGNN
RONWLD AILSOR AEAGNN
RONALD WILSON REAGAN
```

And here is Jeremy's code to generate those strings:

```
/*
*****
* Descramble.c
* A procedure that returns a smooth
```

### Showcase



**ChronoSync 4.8**

Synchronize & Backup Like Clockwork

### THIS MONTH'S ISSUE



**Issue 353**

*Code Debugging and  
Spelunking*

Check out the latest including:  
 x86 Code Debugging and  
 Spelunking ; An Introduction  
 to WebGL ; Managing ESXi  
 from your Mac ; Deploying  
 SHA-3 ; An introduction to  
 Julia ; KoolTools: BBEEdit 11.5,  
 TextSoap For OS X, Honeywell  
 Lyric Round Wi-Fi Thermostat,  
 My Cloud EX2 Ultra ; MacTech  
 Spotlight: Daniel H Steinberg




**Lost Data?**

**We Can Save it!**

- Fast
- Reliable
- Certified
- Secure

800.440.1904  
[www.drivesavers.com](http://www.drivesavers.com)

### TODAY'S DEAL

```

* transition from one string to
* another with the same characters,
* but in a different order.
*****/

#include <stdlib.h>
#include <string.h>

void Descramble(
    Str255 startString,
    Str255 endString,
    unsigned short numSteps,
    Str255 *stepStringPtrs[20] )
{
    short i, j, k;
    short strLength = startString[0];
    div_t divResult;
    short destination, memSwitch;
    BooleanswitchPossible;

    short distances[256];
    short destMap[256];
    BooleandestSwitchMap[256];
    BooleancheckList[256];

    /* Clear the destination switching
     * map for use. Only clear it up to
     * strLength to save time. */
    for (i = 1; i <= strLength; i++)
        destSwitchMap[i] = false;

    /* This loop searches for a duplicate
     * char 'j' for the char 'i'. Once it * finds a duplicate, it checks
to
     * see whether it is already being
     * used as another char's
     * destination. If not, it shows that
     * char 'j' is char 'i's destination,
     * sets the destMap to show that char
     * 'j' is being used, and
     * calculates how far the char 'i' needs to travel on
     * each step to get to its destination. */
    for (i = 1; i <= strLength; i++)
        for (j = 1; j <= strLength; j++)
            if ( startString[i] == endString[j] &&
                ! destSwitchMap[j] )
            {
                /* If destSwitchMap[j] is set, then it already has a
                 * char which is using it for a destination. */
                destSwitchMap[j] = true;

                /* Remember this char's dest. */
                destMap[i] = j;

                /* Find out how far this char should move
                 * on each step. */
                if (i == j)
                    distances[i] = 0;
                else
                {
                    divResult = div(j - i, numSteps);
                    distances[i] = divResult.quot;

                    /* Use the remainder to make
                     * sure this char moves each step.*/
                    if (divResult.rem != 0)
                        if (divResult.rem < 0)
                            distances[i] -= 1;
                    else
                        distances[i] += 1;

                    /* Increment to exit loop. */
                    j = 512;
                }
            }
}

```

## ChronoSync 4.8



Synchronize & Backup Like Clockwork

### Search

#### Community Search:

GO

#### MacTech Search:

GO

### Software Updates via MacUpdate



Remember  
when typing  
felt good?

### Latest Forum Discussions

SEE ALL

## iPHONE APP SHOWCASE

### Price Scanner via MacPrices.net

### Jobs Board

```

}
}

/* In this loop, each character tries to move towards its
 * destination. Its distance is then recalculated to
 * compensate for it being switched. This creates a
 * 'morphing' effect, where the letters in the start string
 * gradually change to their positions in the end string. */
for (i = 0; i < numSteps; i++)
{
    /* Copy the appropriate string for switching. */
    if (i > 0)
        memcpy(*stepStringPtrs[i],
            *stepStringPtrs[i - 1],
            strLength + 1);
    else
        memcpy(*stepStringPtrs[0], startString,
            strLength + 1);

    /* Clear the check list for use. */
    for (k = 1; k <= strLength; k++)
        checkList[k] = false;

    /* This loop switches characters until
     * switchPossible = false. */
    do
    {
        switchPossible = false;

        for (j = 1; j <= strLength; j++)
        {
            if (distances[j] != 0 &&
                ! checkList[j])
            {
                /* Calculate this char's intended
                 * destination for this step. */
                destination = j + distances[j];

                if (checkList[destination])
                {
                    /* If the destination has already been used, find
                     * the nearest one to it to switch to. */
                    destination = -1;

                    if (distances[j] > 0)
                        for (k = destination - 1; k > j;
                            k -= 1)
                            if (! checkList[k] &&
                                distances[k] != 0)
                            {
                                destination = k;
                                k = 512;
                            }
                        else;
                    else
                        for (k = destination + 1; k < j;
                            k++)
                            if (! checkList[k] &&
                                distances[k] != 0)
                            {
                                destination = k;
                                k = 512;
                            }
                }

                if (destination > 0)
                {
                    /* If destination is a valid number, do the
                     * necessary switching. Switch the characters
                     * in the string */
                    memSwitch =
                        stepStringPtrs[i][0][destination];
                    stepStringPtrs[i][0][destination]

```

```

= stepStringPtrs[i][0][j];
    stepStringPtrs[i][0][j] =
memSwitch;

    /* Switch the character's mapped destinations. */
    memSwitch = destMap[destination];
    destMap[destination] = destMap[j];
    destMap[j] = memSwitch;

    /* Show that the destination has been switched. */
    checkList[destination] = true;

    /* Recalculate the switched char's distance. */
    if (destMap[j] == j)
        distances[j] = 0;
    else if (i + 1 < numSteps)
    {
        divResult = div(destMap[j] - j,
numSteps - i - 1);
        distances[j] = divResult.quot;
        if (divResult.rem != 0)
            if (divResult.rem < 0)
                distances[j] -= 1;
        else
            distances[j] += 1;
    }

    switchPossible = true;
}
}
}
} while (switchPossible);

if (i + 1 == numSteps)
return;

/* Recalculate all distances to compensate
 * for switched char's. */
for (k = 1; k <= strLength; k++)
if (distances[k] != 0)
{
    if (destMap[k] == k)
        distances[k] = 0;
    else if (i + 1 < numSteps)
    {
        /* Recalculate how far this char needs to move
 * with the number of steps that are left. */
        divResult = div(destMap[k] - k,
numSteps - i - 1);
        distances[k] = divResult.quot;
        if (divResult.rem != 0)
            if (divResult.rem < 0)
                distances[k] -= 1;
        else
            distances[k] += 1;
    }
}
}
}
}
}

```

## Rules

Here's how it works: Each month there will be a different programming challenge presented here. First, you must write some code that solves the challenge. Second, you must optimize your code (a lot). Then, submit your solution to MacTech Magazine (formerly MacTutor). A winner will be chosen based on code correctness, speed, size and elegance (in that order of importance) as well as the postmark of the answer. In the event of multiple equally desirable solutions, one winner will be chosen at random (with honorable mention, but no prize, given to the runners up). The prize for the best solution each month is \$50 and a limited edition "The Winner! MacTech Magazine Programming Challenge" T-shirt (not to be found in stores).

*In order to make fair comparisons between solutions, all solutions must be in ANSI compatible C. All entries will be tested with the FPU and 68020 flags turned off in THINK C. When timing routines, the latest version of THINK C will be used (with ANSI Settings plus "Honor 'register' first" and "Use Global Optimizer" turned on) so beware if you optimize for a different C compiler.*

*The solution and winners for this month's Programmers' Challenge will be published in the issue two months later. All submissions must be received by the 10th day of the month printed on the front of this issue.*

*All solutions should be marked "Attn: Programmers' Challenge Solution" and sent to Xplain Corporation (the publishers of MacTech Magazine) via "snail mail" or preferably, e-mail - AppleLink: MT.PROGCHAL, Internet: progchallenge@xplain.com, and CompuServe: 71552,174. If you send via snail mail, please include a disk with the solution and all related files (including contact information). See page 2 for information on "How to Contact Xplain Corporation."*

*MacTech Magazine reserves the right to publish any solution entered in the Programming Challenge of the Month and all entries are the property of MacTech Magazine upon submission. The submission falls under all the same conventions of an article submission.*

SPREAD THE WORD:  [Slashdot](#)  [Digg](#)  [Del.icio.us](#)  [Reddit](#)  [Newsvine](#)

Generate a short URL for this page: [GET LINK](#)

MacTech Magazine. [www.mactech.com](http://www.mactech.com)

**Toll Free 877-MACTECH, Outside US/Canada: 805-494-9797**

MacTech is a registered trademark of Xplain Corporation. Xplain, "The journal of Apple technology", Apple Expo, Explain It, MacDev, MacDev-1, THINK Reference, NetProfessional, Apple Expo, MacTech Central, MacTech Domains, MacNews, MacForge, and the MacTutorMan are trademarks or service marks of Xplain Corporation. Sprocket is a registered trademark of eSprocket Corporation. Other trademarks and copyrights appearing in this printing or software remain the property of their respective holders. Not responsible for typographical errors.

All contents are Copyright 1984-2011 by Xplain Corporation. All rights reserved. Theme designed by [Icreon](#)